Project Analysis Report

Marijana Čupović

Mart 2026

Sadržaj

1 Uvod **1**

2 Dinamička analiza **1**
2.1 Jedinični testovi 1
2.2 Profajliranje 2

3 Statička analiza **3**
3.1 Stilske provere 3
3.2 Ciklomatska složenost 3
3.3 Bezbdnosna provera 4

1 Uvod

Ovaj dokument predstavlja pregled analize biblioteke *Jansson* za predmet Verifikacija softvera na master studijama na Matematičkom fakultetu. Cilj projekta je da se pokaže znanje iz oblasti statičke i dinamičke verifikacije softvera primenom alata na projekat otvorenog koda. Alati koji su primenjeni na ovu biblioteku su *Valgrind, lizard, clang-format, flawfinder*, kao i testiranje jedinica koda korišćenjem *CUnit* biblioteke.

2 Dinamička analiza

2.1 Jedinični testovi

Projekat *Jansson* dolazi sa svitom testova koji jako dobro pokrivaju izvorni kod. Pregled pokrivenosti izvornih fajlova je dat na slici 1, gde se može videti da najmanju pokrivenost imaju fajlovi `dtoa.c` i `hashtable_seed.c` i u kontekstu linija i funkcija.

Izvorni fajl `dtoa.c` sadrži kod za prevođenje `double` vrednosti u niske. Iz izveštaja se vidi da su samo 2 (`dtoa_r` i `nrv_alloc`) od 32 funkcije pokrivene testovima. Da bi se povećala pokrivenost funkcija, kreirani su testovi za funkcije `char * dtoa(double dd, int mode, int ndigits, int *decpt, int *sign, char **rve)` koji se

Filename	Line Coverage				Function Coverage		
	Rate		Total	Hit	Rate	Total	Hit
dtoa.c		10.6 %	1895	201	6.2 %	32	2
dump.c		96.5 %	255	246	100.0 %	14	14
error.c		100.0 %	35	35	100.0 %	4	4
hashtable.c		99.4 %	172	171	100.0 %	23	23
hashtable_seed.c		72.5 %	40	29	80.0 %	5	4
jansson.h		100.0 %	41	41	100.0 %	14	14
load.c		97.0 %	573	556	100.0 %	32	32
lookup3.h		79.4 %	97	77	100.0 %	1	1
memory.c		92.2 %	51	47	100.0 %	8	8
pack_unpack.c		99.6 %	515	513	100.0 %	21	21
strbuffer.c		95.8 %	48	46	100.0 %	8	8
strconv.c		91.9 %	74	68	100.0 %	4	4
utf.c		89.8 %	88	79	100.0 %	5	5
value.c		97.6 %	587	573	100.0 %	84	84
version.c		100.0 %	7	7	100.0 %	2	2

Slika 1: Inicijalna pokrivenost izvornih fajlova

nalaze u fajlu `test_dtoa.c`. Testovi su pisani uz podršku biblioteke `CUnit` sa ciljem da se proveri ispravnost konvertovanja brojeva koji su na granicama opsega i sa različitim nivoima preciznosti prilikom prevođenja. Dodati testovi povećavaju pokrivenost funkcija sa 6.2% na 59.4% (slika 2). Iako ovo nije željena pokrivenost treba imati u vidu da mnoge funkcije neće ni biti korišćene u samoj *jansson* biblioteci jer je ovaj fajl preuzet od Davida M. Gaja. Novi testovi su fokusirani na vrednosti koje mogu da dovedu do prekoračenja ili podkoračenja, kao i na različite preciznosti s kojima se konverzija radi.

	Coverage	Total	Hit
Lines:	32.9 %	1895	623
Functions:	59.4 %	32	19
Branches:	-	0	0

Slika 2: Pokrivenost falja `dtoa.c` novim testovima

Kada je u pitanju fajl `hashtable_seed.c` jednina funkcija koja nije pokrivena ni jednim testom je `seed_from_timestamp_and_pid` koja se poziva ako nisu dostupni ni `urandom` ni `CryptoAPI` na `Windows` sistemima. Ona se koristi kao fallback funkcija ako standardne funkcije nisu dostupne, pa će njeno testiranje biti preskočeno pod hrabrnom pretpostavkom da nikada neće biti pozvana.

2.2 Profajliranje

Prilikom profjaliranja projekta *Jansson* se stavio akcenat na baratanje memorijom uz pomoć alata `memcheck` platforme *Valgrind*. Projekat koristi omotač funcije za osnovne funkcije za rad sa memorijom, pa se može desiti curenje memorije ili nepravilno ili nepotpuno uparivanje funkcija `malloc/realloc` i `free`. Pokretanjem skripte `run_memcheck.sh` se pokreću testovi s kojima biblioteka dolazi uz `memcheck` alat koji prati curenje memorije i neuparene pozive funkcije `free`. Na kraju izvršavanja testa se generiše fajl u kome je izveštaj alata `memcheck` za taj test, kao na slici 3. Analizom tih izlaznih fajlova uočavamo da ni jedan test ne izaziva curenje memorije.

```
==117003== Memcheck, a memory error detector
==117003== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==117003== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==117003== Command: ./test_number
==117003== Parent PID: 116899
==117003==
==117003==
==117003== HEAP SUMMARY:
==117003==     in use at exit: 0 bytes in 0 blocks
==117003==   total heap usage: 34 allocs, 34 frees, 4,082 bytes allocated
==117003==
==117003== All heap blocks were freed -- no leaks are possible
==117003==
==117003== For lists of detected and suppressed errors, rerun with: -s
==117003== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Slika 3: Izveštaj alata `memcheck` za `test_number`

3 Statička analiza

3.1 Stilske provere

Stilska analiza projekta je rađena alatom `clang-format` sa ciljem da se uoče nekonzistentnosti u formatiranju koda, kao i bi se proverilo postojanje grešaka u kucanju ili neka unapređenja koda koja mu ne manjju semantiku. Pokretanjem stripte `rum_make_diffs` se kreira fajl `clang-format-report.txt` u kome se nalazi izveštaj o tome da li postoje izmene koje treba primeniti i ako postoje koliko njih. Dodatno se generišu `.formatted` fajlovi koji predstavljaju verzije izvornih fajlova na koje su primenjena pravila formatiranja konvencije koja je zadata kao argument skripte ili *Google* konvencija ako ni jedna nije zadata. Pored formatiranih fajlova se generišu i `.diff` fajlovi u kojima su prikazane razlike između originalnog koda i formatiranog. Predložene izmene se odnose na nazubljivanje koda (slika 4), pozicije `return` naredbe, poravnanje argumenata i slične stilske pojave.

```
 struct key_len {
-    const char *key;
-    int len;
+   const char *key;
+   int len;
 };
```

Slika 4: Primer izmenjenog nazubljivanja u fajlu `dump.c`

3.2 Ciklomatska složenost

Alatom `lizard` je analizirana ciklomatska složenost biblioteke na osnovu njenog izgleda. Ovo znači da se neće računati stvarna složenost i neke funkcije neće biti vidljivi ukoliko se u njima koristi `#ifndef-#endif` koji nisu aktivni, tj one koje `lizard` ne prepoznaje. Ovo možda izgleda kao veliko ograničenje alta, ali nam on

ipak daje zanimljiv pregled funkcija. Pokretanjem skripte `run_lizard.sh` se pokreće alat i generiše `.html` fajl sa prikupljenim podacima, kao i izveštaj alata. Na osnovu generisanog izveštaja vidimo da funkcije imaju jako veliku ciklomatsku složenost, prvih 5 je dato na slici 5.

File	Function name	Cyclomatic complexity (15)	LOC (1000000)	Token count	Parameter count (100)
../jansson/src/dtoa.c	gethex	1068	2724	17306	4
../jansson/src/dump.c	do_dump	51	161	1158	6
../jansson/src/pack_unpack.c	unpack	46	120	733	3
../jansson/src/load.c	lex_scan_string	42	134	751	2
../jansson/src/pack_unpack.c	unpack_object	30	104	611	3

Slika 5: Prvih 5 funkcija sa najvećom ciklomatskom složenošću

Najsloženija je funkcija `gethex` sa 2724 linije koda i ciklomatskom složenošću 1068. Ako pogledamo njen izvorni kod postaje nam jasno odakle potiče ovolika složenost. Ona pretvara nisku koja predstavlja heksadecimalan broj u `long long`, pazeći na obradu specijalnih slučajeva propisanih standardnom *IEEE 754*. Ova funkcija ima dosta udnježdenih `switch` naredbi u kojima se pojavljuju grananja i petlje, pa je jasno poreklo velike složenosti ove funkcije.

Pregledom izveštaja po fajlovima koji je prikazan na slici 6, vidimo da je najveća prosečna ciklomatična složenost baš u fajlu `dtoa.c` koji sadrži funkciju `gethex`. Kod ostalih fajlova je prosečna ciklomatska složenost značajno manja (od 2.0 do 9.3). Ova velika razlika ima smisla ako se setimo da je `dtoa.c` preuzeta od Dejvida M. Gaja koji je kreirao ovaj kod kako bi rešio problem konverzije brojeva u pokretnom zarezu u niske i to efikasno i precizno i nezavisno od platforme.

```
13 file analyzed.
==============================================================
NLOC    Avg.NLOC  AvgCCN  Avg.token  function_cnt    file
--------------------------------------------------------------
   4601     137.3    50.2      882.5        26     ../jansson/src/dtoa.c
    363      24.6     7.9      168.9        14     ../jansson/src/dump.c
     48      11.5     2.8       83.0         4     ../jansson/src/error.c
    237       9.8     2.3       71.6        23     ../jansson/src/hashtable.c
    154      14.8     5.0       85.8         9     ../jansson/src/hashtable_seed.c
    814      23.8     6.6      144.6        32     ../jansson/src/load.c
     69       7.8     2.8       41.8         8     ../jansson/src/memory.c
    718      32.9     9.3      201.2        21     ../jansson/src/pack_unpack.c
     61       7.1     2.0       47.9         8     ../jansson/src/strbuffer.c
    160      25.0     6.7      156.0         6     ../jansson/src/strconv.c
    113      22.2     9.6      156.4         5     ../jansson/src/utf.c
    801       9.2     2.9       56.6        86     ../jansson/src/value.c
     12       5.5     2.0       30.0         2     ../jansson/src/version.c
```

Slika 6: Izvešta o broju linija i složenosti po izvornim fajlovima

3.3 Bezbdnosna provera

Kako bi se otkrile potencijalni beybednosti propusti koji nastaju upotrebom funkcija za koje se dobro zna da imaju probleme, korišćen je alat `flawfinder`. On radi prosto poređenje teksta koda sa nazivima funkcija iz ugrađenje baze ranjivih funkcija, ignorišući komentare i niske. Kako alat ne razmatra semantiku programa, analizi rezultata treba pristupiti kritički i usvojiti otkrivene ranjivosti

samo ako one imaju smisla u kontekstu u kom se nalaze. Pokretanjem skripte `run_flawfinder.sh` se pokreće alat nad fajlovima iz `jansson/src` direktorijuma i generiše `.html` fajl koji sadrži spisak pronađenih ranjivosti zajedno sa kontekstom u kome se nalaze problematične linije. Alat je našao 74 potencijalne ranjivosti čija je raspodela data u tabeli 1.

Nivo	0	1	2	3	4	5
Broj pogodaka	8	22	36	0	16	0

Tabela 1: Raspodela potencijalnih ranjivosti po nivoima

Nivoi se odnose na stepen rizika koji dolazi sa upotrebom određenih funkcija. On zavisi od same funkcije, ali i od vrednosti parametara koje `flawfinder` proverava i procenjuje nivo rizika. Ova procena je jako naivna jer alat nema svest o kontroli toka programa niti o tipovima paramatara.

Prolaskom kroz generisani izveštaj uočavamo da su rizične funkcije pretežno one za pisanje (`snprint, vsnprint, printf`), rad sa niskama, a rizičnom se smatra i upotreba statičkih nizova. Statički nizovi ne predtsvljaju problem u ovom kodu jer se ili zna tačna veličina prilikom kompilacije ili postoje provere granica. Što se tiče upozorenja vezanih za nebezbedne funkcije, neka upozorenja se odnose na fajl `dtoa.c` koji je napisan 1990−ih godina kada još nisu postojale *bezbedne* verzije funkcija. Kada je reč o stalim datotekama, koje su pisane kasnije, funkcije se mogu zameniti svojim bezbednima pandanima, ali su autori verovatno zadržali originalne funkcije kako bi održali maksimalnu prenosivost, a o bezbednosti ručno paze. Zanimljivo je što za svako upovorenje u listi postoji link ka *Common Weakness Enumeration* sajtu gde se nalazi opis ranjivosti navedene funkcije.